



Assignment-5

Aim:- Develop C functions to insert and delete into from a max heap under assumption that dynamically allocated array is used, the initial capacity of this array is 1, and array doubling is done whenever we are to insert into a max heap that is full.

Algorithm:-

1. Create heap steps:

- ① Allocate memory for a Maxheap structure heap.
- ② If allocation fails, display error and terminate.
- ③ Set heap.capacity = 1
- ④ Set heap.size = 0;
- ⑤ Allocate memory for array heap.arr of size (heap.capacity * sizeof(int)).
- ⑥ If array allocation fails, free heap structure and terminate.
- ⑦ Return heap pointer.

2. Insert Algorithm with Array Doubling (insert)

- ① check if heap is full (heap.size == heap.capacity):
 - ① Set heap.capacity = heap.capacity * 2
 - ② Reallocate heap.arr with new capacity using realloc
 - ③ If reallocation fails, print error and exit function.
- ② Place 'value' at the end of the heap array:
: heap.arr[heap.size] = value.



③ Set $index = heap.size$

④ Up heapify :

while($i > 0$ & $heap.arr[(i-1)/2] < heap.arr[i]$:

① Swap $heap.arr[i]$ with parent $heap.arr[(i-1)/2]$

② Update $i = (i-1)/2$.

⑤ Return success.

3. Delete Max

① check for underflow i.e if $heap.size \leq 0$
print error and return -1;

② store the root element $root = heap.arr[0]$;

③ Move the last element to the root position
 $heap.arr[0] = heap.arr[heap.size - 1]$

④ decrement $heap.size$ by 1.

⑤ Set index $i = 0$.

⑥ Down-heapify

while (true)

① Set $left = 2 * i + 1$

② Set $right = 2 * i + 2$

③ Set $largest = i$

④ If $left < heap.size$ And $heap.arr[left] > heap.arr[largest]$
Set $largest = left$.

⑤ If $right < heap.size$ And $heap.arr[right] > heap.arr[largest]$
Set $largest = right$.

⑥ If $largest == i$
Breakout of loop ;

⑦ Swap $heap.arr[i]$ with $heap.arr[largest]$

⑧ update $i = largest$.

return root;



4. print heap

a) If $\text{heap.size} == 0$,
print "heap is empty" and return

b) for i from 0 to $\text{heap.size} - 1$
print $(\text{heap.array}[i])$.

c) Print current size (heap.size) and current capacity (heap.capacity).

Conclusions: We have successfully Implemented the Max heap insertion and deletion in the Max heap. Overall The Insert can take upto $O(\log n)$ and deletion can also take upto $O(\log n)$. Also we have used dynamically allocated array.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {
```

```
    int *arr;    // dynamically allocated array
```

```
    int size;    // current number of elements
```

```
    int capacity; // total available capacity
```

```
} MaxHeap;
```

```
// initialize the heap with initial capacity = 1
```

```
MaxHeap* createHeap() {
```

```
    MaxHeap *heap = (MaxHeap *)malloc(sizeof(MaxHeap));
```

```
    if (!heap) {
```

```
        printf("Memory allocation failed for heap structure.\n");
```

```
        exit(1);
```

```
    }
```

```
    heap->capacity = 1;
```

```
    heap->size = 0;
```

```
    heap->arr = (int *)malloc(heap->capacity * sizeof(int));
```

```
    if (!heap->arr) {
```

```
        printf("Memory allocation failed for heap array.\n");
```

```
        free(heap);
```

```
        exit(1);
```

```
    }
```

```
    return heap;
```

```
}
```

```
// insert function using array doubling
```

```
void insert(MaxHeap *heap, int value) {
```

```

if (heap->size == heap->capacity) {
    heap->capacity *= 2;
    int *temp = (int *)realloc(heap->arr, heap->capacity * sizeof(int));
    if (!temp) {
        printf("Reallocation failed!\n");
        return;
    }
    heap->arr = temp;
    printf("-> [Capacity doubled to %d]\n", heap->capacity);
}

```

```

int i = heap->size;
heap->arr[i] = value;
heap->size++;

```

```

// heapify
while (i != 0 && heap->arr[(i - 1) / 2] < heap->arr[i]) {
    int temp = heap->arr[i];
    heap->arr[i] = heap->arr[(i - 1) / 2];
    heap->arr[(i - 1) / 2] = temp;
    i = (i - 1) / 2;
}
printf("Successfully inserted %d\n", value);
}

```

// delete root maximum element

```

int deleteMax(MaxHeap *heap) {
    if (heap->size <= 0) {
        printf("Heap Underflow! Heap is empty.\n");
    }
}

```

```

        return -1;
    }

    int root = heap->arr[0];
    heap->arr[0] = heap->arr[heap->size - 1];
    heap->size--;

    //down heapify
    int i = 0;
    while (1) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int largest = i;

        if (left < heap->size && heap->arr[left] > heap->arr[largest]) {
            largest = left;
        }
        if (right < heap->size && heap->arr[right] > heap->arr[largest]) {
            largest = right;
        }

        if (largest == i) {
            break;
        }

        int temp = heap->arr[i];
        heap->arr[i] = heap->arr[largest];
        heap->arr[largest] = temp;
        i = largest;
    }

```

```
}
```

```
return root;
```

```
}
```

```
// helper function to print array and current status
```

```
void printHeap(MaxHeap *heap) {
```

```
    if (heap->size == 0) {
```

```
        printf("Heap is empty.\n");
```

```
        return;
```

```
    }
```

```
    printf("Heap Array Elements: ");
```

```
    for (int i = 0; i < heap->size; i++) {
```

```
        printf("%d ", heap->arr[i]);
```

```
    }
```

```
    printf("\nSize: %d | Capacity: %d\n", heap->size, heap->capacity);
```

```
}
```

```
// free allocated memory
```

```
void freeHeap(MaxHeap *heap) {
```

```
    if (heap) {
```

```
        free(heap->arr);
```

```
        heap->arr = NULL;
```

```
        free(heap);
```

```
    }
```

```
}
```

```
int main() {
```

```
    MaxHeap *heap = createHeap();
```

```
int choice, val, deletedVal;
```

```
while (1) {
```

```
    printf("\n=== MAX HEAP OPERATIONS ===\n");
```

```
    printf("1. Insert\n");
```

```
    printf("2. Delete Max\n");
```

```
    printf("3. Display Heap\n");
```

```
    printf("4. Exit\n");
```

```
    printf("Enter choice: ");
```

```
    scanf("%d", &choice);
```

```
    switch (choice) {
```

```
        case 1:
```

```
            printf("Enter integer value to insert: ");
```

```
            scanf("%d", &val);
```

```
            insert(heap, val);
```

```
            break;
```

```
        case 2:
```

```
            deletedVal = deleteMax(heap);
```

```
            if (deletedVal != -1) {
```

```
                printf("Deleted Maximum Element: %d\n", deletedVal);
```

```
            }
```

```
            break;
```

```
        case 3:
```

```
            printHeap(heap);
```

```
            break;
```


case 4:

freeHeap(heap);

printf("Exiting program and freeing memory...\n");

return 0;

default:

printf("Invalid choice! Try again.\n");

}

}

}